



Department of Computer Science and engineering(Iot, Cyber security with block chain technology)

Class: T.Y. B.Tech.

Semester: VI

Subject: Big Data Analytics

Course Code:

Mini Project based on Big Data

Project Title: Air Quality Analysis of Delhi Using Big Data Tools

Group Number: 2

Project group's Name, roll number and SAP ID:

Name	Roll Number	SAP ID
Dhriti Vaidya	B023	60019220018
Gargi Sukhatankar	B027	60019220125
Meenakshi Chandra	B039	60019220126

Need for the Project:

Air pollution is a critical environmental issue, especially in metropolitan cities like Delhi. Monitoring air quality and identifying pollution trends can help the government and citizens take preventive actions. This project leverages big data tools to analyze real-time and historical air quality data, enabling better understanding and decision-making regarding pollution control and public health.

Outcomes of the Project:

- **Hands-on Experience with Big Data Tools:** The project offers practical exposure to big data technologies like Apache Spark and PySpark, helping students understand how large-scale datasets can be processed efficiently. It builds foundational knowledge in distributed computing, data frames, and schema inference.
- **Improved Understanding of Environmental Data Analysis:** By working with real-world air quality datasets, users gain a deeper understanding of environmental monitoring. They learn how to identify pollution trends, critical pollutants, and geographical patterns through statistical and visual analysis.
- **Proficiency in Data Cleaning and Preprocessing:** The project emphasizes the importance of preprocessing, including handling null values, converting data types, and filtering invalid entries. These are crucial steps in ensuring data quality and accuracy during analysis.
- **Mastery of Grouping, Aggregation, and Trend Analysis:** Students learn how to group large volumes of data by location and time, compute averages and maximum values, and identify seasonal or location-based pollution patterns—core concepts in data analytics.
- **Practical Visualization Skills Using Python Libraries:** The use of libraries like matplotlib and seaborn introduces students to powerful visualization techniques. They learn how to build readable and insightful charts such as bar plots, line graphs, and heatmaps to communicate findings effectively.
- **Scalable Data Processing Knowledge:** Unlike basic data analysis tools, Spark enables processing of massive datasets with high speed. Students explore the scalability benefits of Spark and understand how big data frameworks outperform traditional tools for large datasets.
- **Integration of Pandas and PySpark Workflows:** The project bridges the gap between big data (PySpark) and traditional data science (Pandas), giving students a flexible approach to data transformation and analysis. This hybrid skillset is valuable in real-world analytics jobs.



Department of Computer Science and engineering(Iot, Cyber security with block chain technology)

Class: T.Y. B.Tech.

Semester: VI

- **Awareness of Air Quality and Environmental Impact:** Beyond technical skills, the project raises awareness about air pollution, helping students recognize the importance of analyzing and improving air quality in urban settings. It links data science to real-world societal problems.
- **Foundation for Advanced Predictive Modeling:** The output from this project can serve as a baseline for future work, such as time-series forecasting, AQI prediction using machine learning models, or anomaly detection. It paves the way for innovation in environmental intelligence systems.
- **Inspiration for Data-Driven Environmental Solutions:** By engaging with real datasets and producing meaningful insights, students are inspired to explore how data science can contribute to sustainable development. This could lead to new ideas in smart cities, IoT-based monitoring, or policy analysis.

Technologies Used:

- **Big Data Framework:** Apache Spark (PySpark)
- **Programming Language:** Python
- **Data Visualization:** Matplotlib, Seaborn
- **Data Processing:** Pandas, Spark DataFrame API
- **Machine Learning (Optional):** MinMaxScaler from Scikit-learn (if normalization or prediction is done)

Software & Hardware Requirements:

Software:

- Jupyter Notebook / VS Code
- Apache Spark with PySpark
- Python 3.8+
- Required libraries: pandas, pyspark, matplotlib, seaborn, sklearn

Hardware:

- Minimum 4GB RAM (8GB recommended)
- i3 processor or higher
- Operating System: Windows/Linux/Mac

Project Description:

The project focuses on analyzing air quality data from Delhi using Big Data techniques. The dataset includes parameters like AQI, date, pollutant levels (PM2.5, NO2, etc.), and locations. Using PySpark, the data is cleaned, transformed, and explored to uncover trends and patterns. Visualizations help in interpreting these insights clearly.

Steps to Run the Project:

1. Install Apache Spark and PySpark.
2. Set up Jupyter Notebook or any Python IDE.
3. Load the dataset (aqi_delhi.csv).
4. Perform data cleaning (handle nulls, format dates).
5. Analyze the AQI distribution across locations and months.
6. Use visualizations to show trends.
7. (Optional) Apply normalization or ML models.



Department of Computer Science and engineering(Iot, Cyber security with block chain technology)

Class: T.Y. B.Tech.

Semester: VI

Project code with explanation:

Code:

```
!https://www.kaggle.com/datasets/deepaksirohiwal/delhi-air-quality
!pip install kaggle

Requirement already satisfied: kaggle in /usr/local/lib/python3.11/dist-packages (1.7.4.2)
Requirement already satisfied: bleach in /usr/local/lib/python3.11/dist-packages (from kaggle) (6.2.0)
Requirement already satisfied: certifi>=14.05.14 in /usr/local/lib/python3.11/dist-packages (from kaggle) (2025.1.31)
Requirement already satisfied: charset-normalizer in /usr/local/lib/python3.11/dist-packages (from kaggle) (3.4.1)
Requirement already satisfied: idna in /usr/local/lib/python3.11/dist-packages (from kaggle) (3.10)
Requirement already satisfied: protobuf in /usr/local/lib/python3.11/dist-packages (from kaggle) (5.29.4)
Requirement already satisfied: python-dateutil>=2.5.3 in /usr/local/lib/python3.11/dist-packages (from kaggle) (2.8.2)
Requirement already satisfied: python-slugify in /usr/local/lib/python3.11/dist-packages (from kaggle) (8.0.4)
Requirement already satisfied: requests in /usr/local/lib/python3.11/dist-packages (from kaggle) (2.32.3)
Requirement already satisfied: setuptools>=21.0.0 in /usr/local/lib/python3.11/dist-packages (from kaggle) (75.2.0)
Requirement already satisfied: six>=1.10 in /usr/local/lib/python3.11/dist-packages (from kaggle) (1.17.0)
Requirement already satisfied: text-unidecode in /usr/local/lib/python3.11/dist-packages (from kaggle) (1.3)
Requirement already satisfied: tqdm in /usr/local/lib/python3.11/dist-packages (from kaggle) (4.67.1)
Requirement already satisfied: urllib3>=1.15.1 in /usr/local/lib/python3.11/dist-packages (from kaggle) (2.3.0)
Requirement already satisfied: webencodings in /usr/local/lib/python3.11/dist-packages (from kaggle) (0.5.1)

!pip install pyspark

Requirement already satisfied: pyspark in /usr/local/lib/python3.11/dist-packages (3.5.5)
Requirement already satisfied: py4j==0.10.9.7 in /usr/local/lib/python3.11/dist-packages (from pyspark) (0.10.9.7)

[2] # Import necessary libraries
from pyspark.sql import SparkSession
from pyspark.sql.functions import col, mean, max, min, count, to_date
import matplotlib.pyplot as plt
import seaborn as sns

[3] # Create a Spark session
spark = SparkSession.builder.appName("delhi_aqi").getOrCreate()

[4] # Set dataset path (update this with the actual path)
dataset_path = "delhi_aqi.csv"
```



SHRI VILEPARLE KELAVANI MANDAL'S
DWARKADAS J. SANGHVI COLLEGE OF ENGINEERING
(Autonomous College Affiliated to the University of Mumbai)
NAAC ACCREDITED with "A" GRADE (CGPA : 3.18)



Department of Computer Science and engineering(Iot, Cyber security with block chain technology)

Class: T.Y. B.Tech.

Semester: VI

```
# Show first 5 rows  
df.show(5)
```

```
+-----+-----+-----+-----+-----+-----+-----+-----+-----+  
|          date|      co|    no|    no2|    o3|    so2| pm2_5| pm10| nh3|  
+-----+-----+-----+-----+-----+-----+-----+-----+-----+  
|2020-11-25 01:00:00|2616.88|  2.18|   70.6|13.59|38.62|364.61|411.73|28.63|  
|2020-11-25 02:00:00|3631.59|23.25|   89.11| 0.33|54.36|420.96|486.21|41.04|  
|2020-11-25 03:00:00|4539.49|52.75|100.08| 1.11|68.67|463.68|541.95|49.14|  
|2020-11-25 04:00:00|4539.49|50.96|111.04| 6.44|78.2|454.81|534.0|48.13|  
|2020-11-25 05:00:00|4379.27|42.92| 117.9|17.17|87.74|448.14|529.19|46.61|  
+-----+-----+-----+-----+-----+-----+-----+-----+-----+  
only showing top 5 rows
```

```
df.head(5)
```

```
[Row(date=datetime.datetime(2020, 11, 25, 1, 0), co=2616.88, no=2.18, no2=70.6, o3=13.59, so2=38.62, pm2_5=364.61, pm10=411.73, nh3=28.63),  
Row(date=datetime.datetime(2020, 11, 25, 2, 0), co=3631.59, no=23.25, no2=89.11, o3=0.33, so2=54.36, pm2_5=420.96, pm10=486.21, nh3=41.04),  
Row(date=datetime.datetime(2020, 11, 25, 3, 0), co=4539.49, no=52.75, no2=100.08, o3=1.11, so2=68.67, pm2_5=463.68, pm10=541.95, nh3=49.14),  
Row(date=datetime.datetime(2020, 11, 25, 4, 0), co=4539.49, no=50.96, no2=111.04, o3=6.44, so2=78.2, pm2_5=454.81, pm10=534.0, nh3=48.13),  
Row(date=datetime.datetime(2020, 11, 25, 5, 0), co=4379.27, no=42.92, no2=117.9, o3=17.17, so2=87.74, pm2_5=448.14, pm10=529.19, nh3=46.61)]
```



✓
0s

```
[8] # Print dataset schema
df.printSchema()
```

```
root
|-- date: timestamp (nullable = true)
|-- co: double (nullable = true)
|-- no: double (nullable = true)
|-- no2: double (nullable = true)
|-- o3: double (nullable = true)
|-- so2: double (nullable = true)
|-- pm2_5: double (nullable = true)
|-- pm10: double (nullable = true)
|-- nh3: double (nullable = true)
```

✓
1s

```
# Get total number of rows
print(f"Total records: {df.count()}")
```

```
Total records: 18776
```

✓
3s

```
# Get summary statistics of numerical columns
df.describe().show()
```

```
summary|          co|          no|          no2|          o3|          so2|          pm2_5|          pm10|
count|      18776|      18776|      18776|      18776|      18776|      18776|      18776|
mean|2929.2286275032225|33.66070195994916| 66.221298998722|60.34623934810406|66.69363282914416|238.13030890498518|300.09296602045157| 25.
stddev|2854.5235056532933|62.12711802202761|48.527491992637245|80.46493196439782|49.43919077821275|226.53362509392468|267.16582664652094|26.
min|      260.35|          0.0|          4.28|          0.0|          5.25|          11.83|          15.07|
max|     21148.68|      500.68|      460.62|      801.09|      579.83|     1708.09|     1969.93|
```

✓
0s

```
[11] df.columns
```

```
['date', 'co', 'no', 'no2', 'o3', 'so2', 'pm2_5', 'pm10', 'nh3']
```



✓
0s

```
[12] # Drop rows where all values are null
df = df.dropna(how="all")
```

✓
4s

```
[13] # Fill missing values with column mean (for numeric data)
numeric_cols = ['pm2_5', 'pm10', 'no2', 'so2', 'co', 'o3']
for col_name in numeric_cols:
    mean_val = df.select(mean(col(col_name))).collect()[0][0]
    df = df.fillna({col_name: mean_val})
```

✓
0s



```
# Verify missing values are handled
df.select([count(col(c)).alias(c) for c in df.columns]).show()
```



date	co	no	no2	o3	so2	pm2_5	pm10	nh3
18776	18776	18776	18776	18776	18776	18776	18776	18776

✓
0s

```
[15] #string to Date type
df = df.withColumn("date", to_date(col("date")))
```

✓
0s

```
[16] # Show transformed data
df.select("date").show(5)
```



date
2020-11-25
2020-11-25
2020-11-25
2020-11-25
2020-11-25

only showing top 5 rows

✓
1s



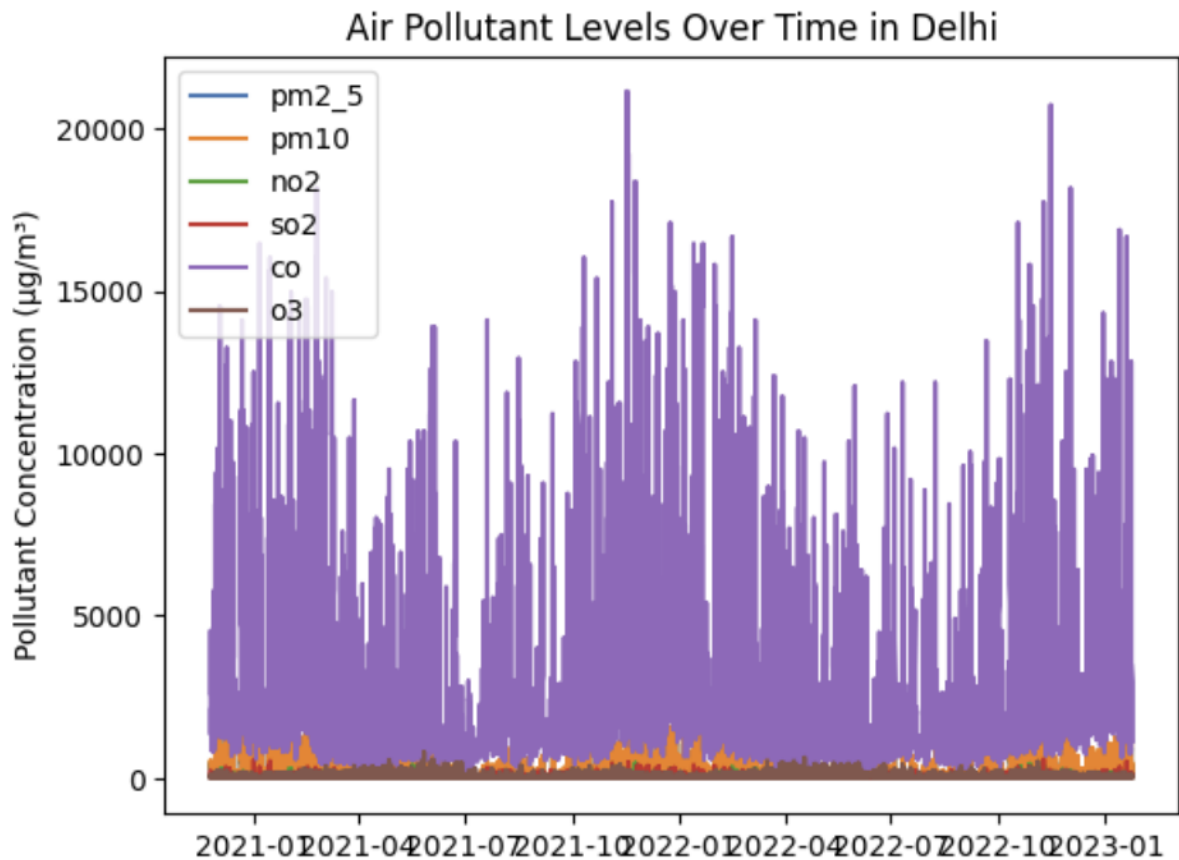
```
# Convert to Pandas for visualization
df_pandas = df.select("date", "pm2_5", "pm10", "no2", "so2", "co", "o3").toPandas()
```



```
[18] # Set date as index  
df_pandas.set_index("date", inplace=True)
```

```
# Plot pollutant trends  
plt.figure(figsize=(12,6))  
df_pandas.plot(kind="line", linewidth=1.5)  
plt.title("Air Pollutant Levels Over Time in Delhi")  
plt.xlabel("Date")  
plt.ylabel("Pollutant Concentration ( $\mu\text{g}/\text{m}^3$ )")  
plt.legend()  
plt.show()
```

<Figure size 1200x600 with 0 Axes>





SHRI VILEPARLE KELAVANI MANDAL'S
DWARKADAS J. SANGHVI COLLEGE OF ENGINEERING
(Autonomous College Affiliated to the University of Mumbai)
NAAC ACCREDITED with "A" GRADE (CGPA : 3.18)



Department of Computer Science and engineering(Iot, Cyber security with block chain technology)

Class: T.Y. B.Tech.

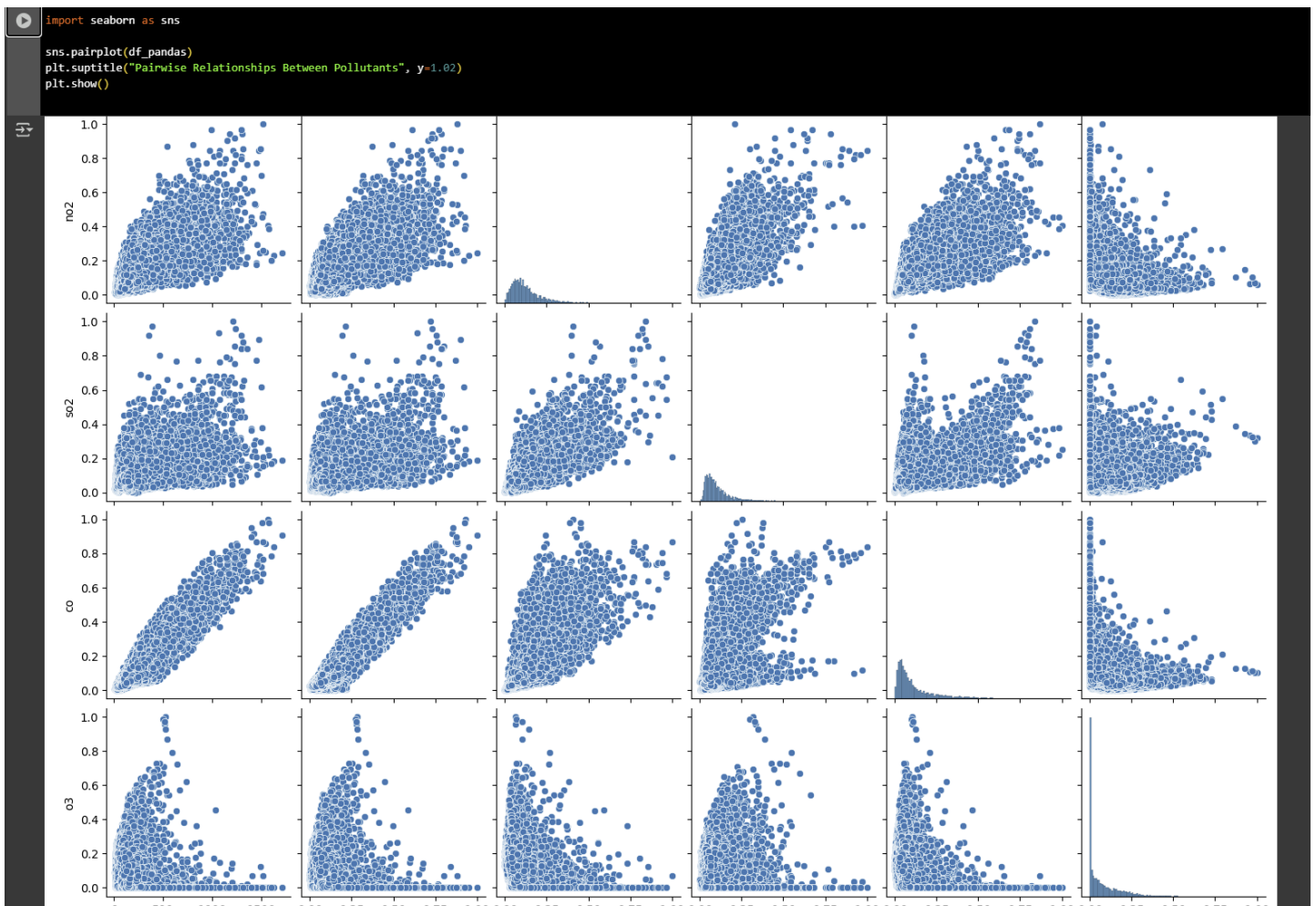
Semester: VI

```
[20] # Compute average pollutant levels
avg_pollutants = df.select([mean(col(c)).alias(c) for c in numeric_cols])
avg_pollutants.show()
```

	pm2_5	pm10	no2	so2	co	o3
	238.13030890498518	300.09296602045157	66.221298998722	66.69363282914416	2929.2286275032225	60.34623934810406

```
#Normalize/Scale pollutant values (If required for ML models)
from sklearn.preprocessing import MinMaxScaler

scaler = MinMaxScaler()
df_pandas.iloc[:, 1:] = scaler.fit_transform(df_pandas.iloc[:, 1:])
```





SHRI VILEPARLE KELAVANI MANDAL'S
DWARKADAS J. SANGHVI COLLEGE OF ENGINEERING
(Autonomous College Affiliated to the University of Mumbai)
NAAC ACCREDITED with "A" GRADE (CGPA : 3.18)

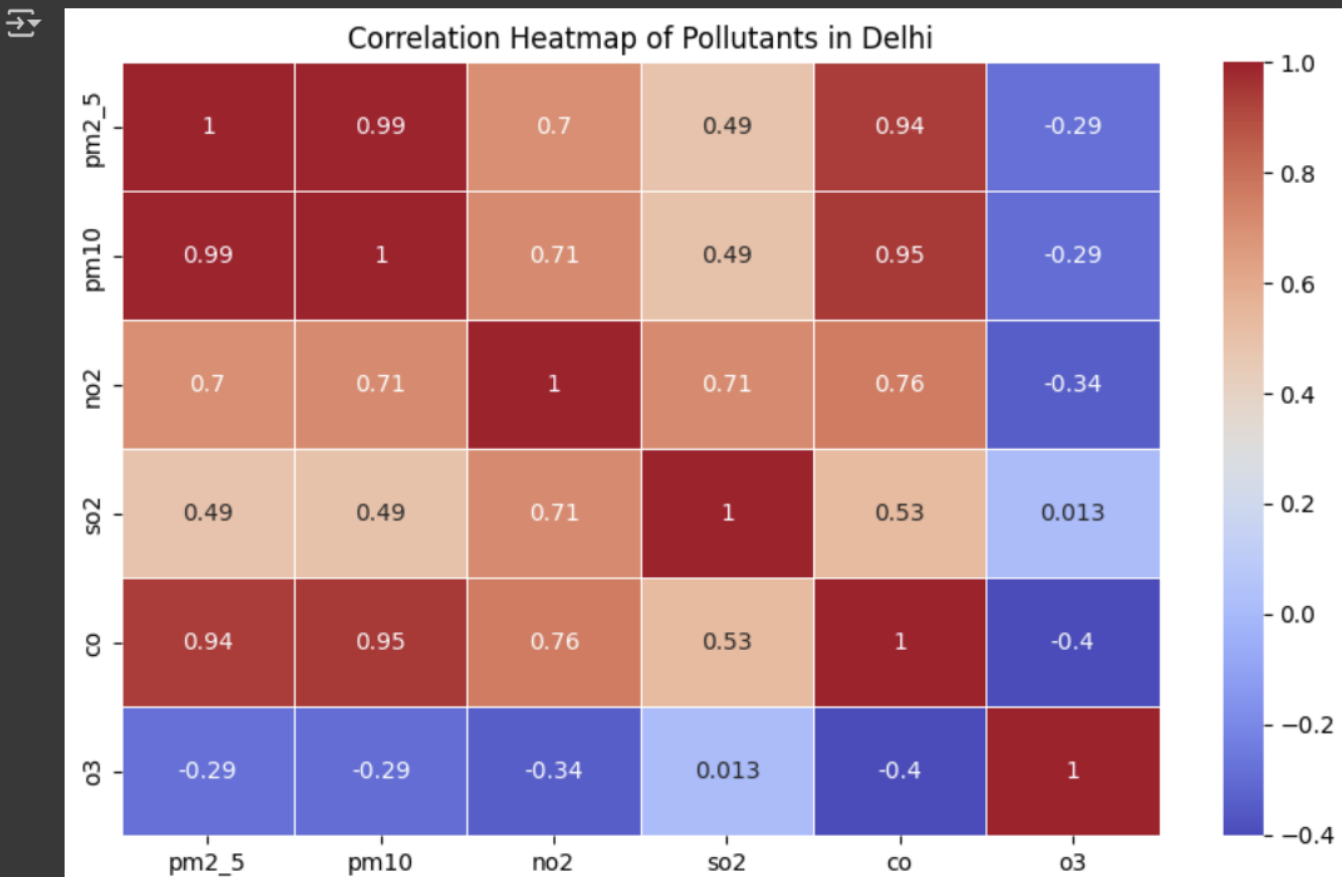


Department of Computer Science and engineering(Iot, Cyber security with block chain technology)

Class: T.Y. B.Tech.

Semester: VI

```
plt.figure(figsize=(10, 6))  
sns.heatmap(df_pandas.corr(), annot=True, cmap="coolwarm", linewidths=0.5)  
plt.title("Correlation Heatmap of Pollutants in Delhi")  
plt.show()
```





Department of Computer Science and engineering(Iot, Cyber security with block chain technology)

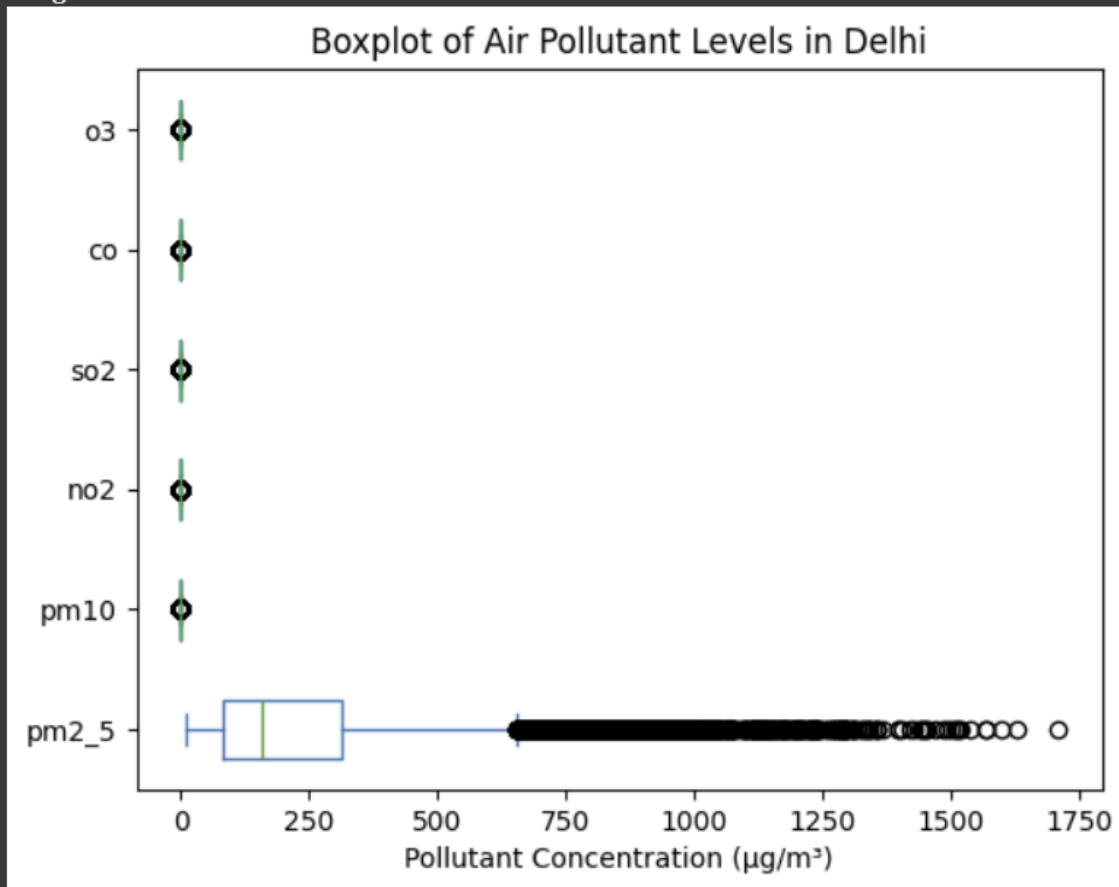
Class: T.Y. B.Tech.

Semester: VI

✓
0s

```
[23] plt.figure(figsize=(12, 6))  
df_pandas.plot(kind="box", vert=False)  
plt.title("Boxplot of Air Pollutant Levels in Delhi")  
plt.xlabel("Pollutant Concentration ( $\mu\text{g}/\text{m}^3$ )")  
plt.show()
```

⇨ <Figure size 1200x600 with 0 Axes>





SHRI VILEPARLE KELAVANI MANDAL'S
DWARKADAS J. SANGHVI COLLEGE OF ENGINEERING
(Autonomous College Affiliated to the University of Mumbai)
NAAC ACCREDITED with "A" GRADE (CGPA : 3.18)

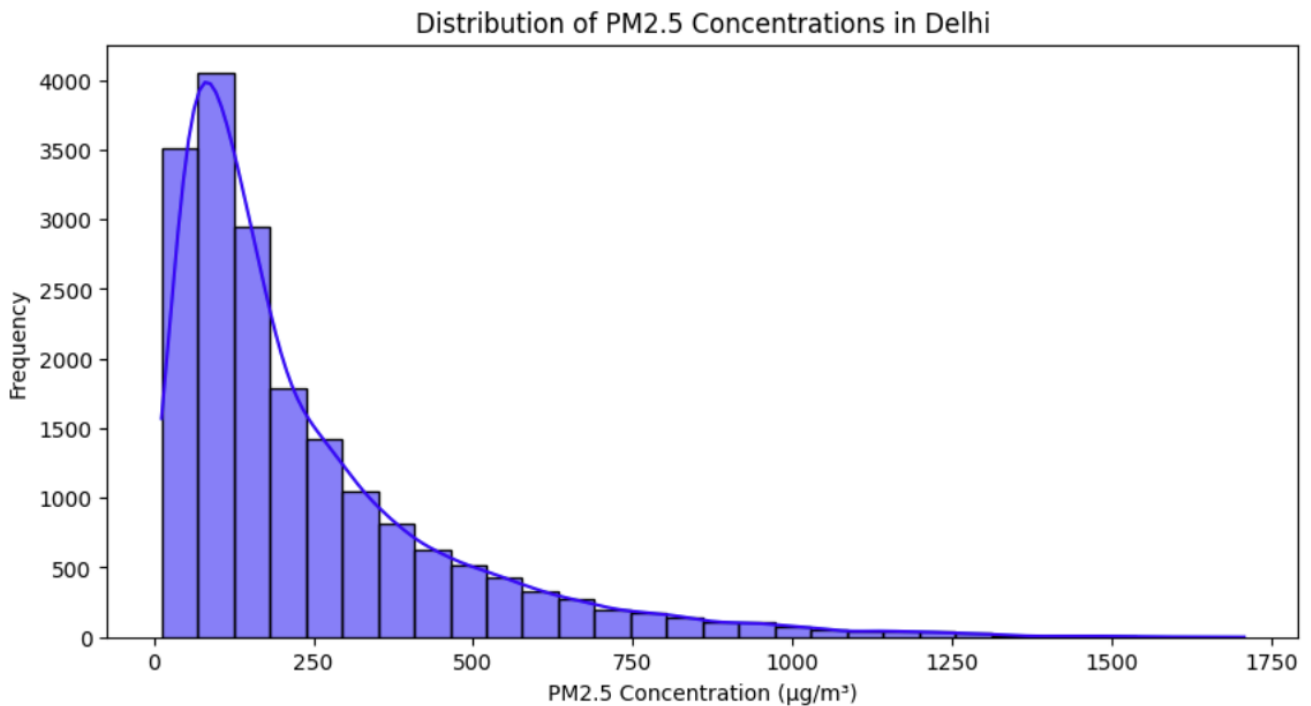


Department of Computer Science and engineering(Iot, Cyber security with block chain technology)

Class: T.Y. B.Tech.

Semester: VI

```
plt.figure(figsize=(10, 5))
sns.histplot(df_pandas["pm2_5"], kde=True, bins=30, color='blue')
plt.title("Distribution of PM2.5 Concentrations in Delhi")
plt.xlabel("PM2.5 Concentration ( $\mu\text{g}/\text{m}^3$ )")
plt.ylabel("Frequency")
plt.show()
```





**SHRI VILEPARLE KELAVANI MANDAL'S
DWARKADAS J. SANGHVI COLLEGE OF ENGINEERING**
(Autonomous College Affiliated to the University of Mumbai)
NAAC ACCREDITED with "A" GRADE (CGPA : 3.18)



Department of Computer Science and engineering(Iot, Cyber security with block chain technology)

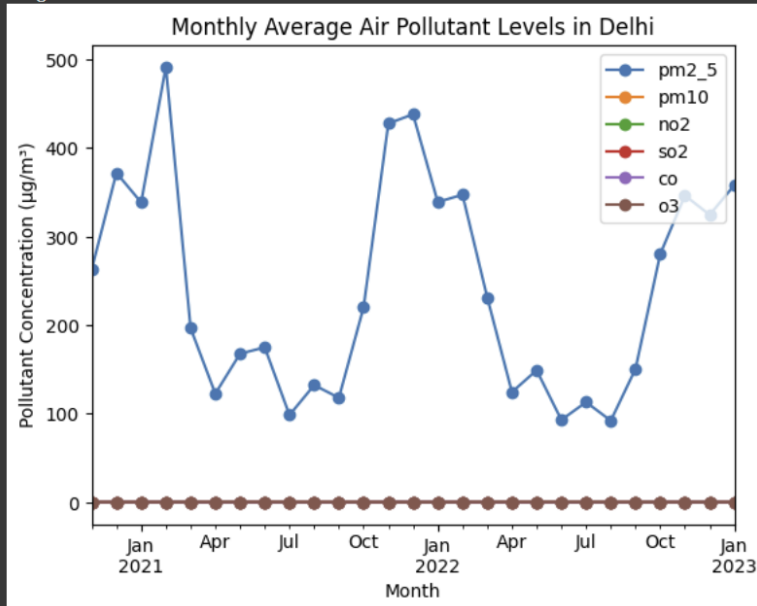
Class: T.Y. B.Tech.

Semester: VI

```
import pandas as pd
df_pandas.index = pd.to_datetime(df_pandas.index) # Ensure date is in datetime format
monthly_avg = df_pandas.resample('M').mean()

plt.figure(figsize=(12, 6))
monthly_avg.plot(marker="o")
plt.title("Monthly Average Air Pollutant Levels in Delhi")
plt.xlabel("Month")
plt.ylabel("Pollutant Concentration (µg/m³)")
plt.legend()
plt.show()
```

```
<ipython-input-26-6c6d046b410d>:3: FutureWarning: 'M' is deprecated and will be removed in a future version, please use 'ME' instead.
monthly_avg = df_pandas.resample('M').mean()
<Figure size 1200x600 with 0 Axes>
```



```
[27] print(df_pandas.isnull().sum()) # Check missing values
```

```
pm2_5    0
pm10     0
no2       0
so2       0
co        0
o3        0
dtype: int64
```

```
[28] df_pandas.fillna(method='ffill', inplace=True) # Forward fill missing values
```

```
<ipython-input-28-4cfff683126a>:1: FutureWarning: DataFrame.fillna with 'method' is deprecated and will raise in a future version. Use obj.ffill() or obj.bfill() instead.
df_pandas.fillna(method='ffill', inplace=True) # Forward fill missing values
```

```
print(df_pandas.duplicated().sum()) # Count duplicate rows
```

```
0
```



Explanation: The project code is structured into three major parts: **data ingestion using PySpark**, **data processing and analysis**, and **visualization of results using Python plotting libraries**. Each segment plays an important role in transforming raw air quality data into actionable insights.

1. Data Ingestion (Using PySpark)

This component focuses on loading and preparing the air quality dataset using PySpark:

- **Spark Session Creation:** A SparkSession is initialized to enable big data processing.
- **Data Loading:** The air quality CSV file is read using `spark.read.csv()` with headers and automatic schema inference.
- **Data Type Handling:** The 'date' column is converted into a proper date format for time-series analysis.

2. Data Processing & Analysis

This part deals with filtering, grouping, and summarizing the data to extract meaningful insights:

- **Filtering Null Values:** Ensures the analysis is based on complete and accurate rows.
- **GroupBy Operations:** The dataset is grouped by location and month to compute average AQI, revealing the most polluted areas and time periods.
- **Ordering Results:** Data is sorted by average AQI to highlight the worst-affected regions.

3. Visualization (Using Matplotlib/Seaborn)

The third segment involves converting Spark DataFrames into Pandas for plotting:

- **Conversion to Pandas:** For advanced visualization, selected Spark DataFrames are converted to Pandas.
- **Line and Bar Charts:** Monthly trends and location-based AQI distributions are shown using matplotlib and seaborn.
- **Heatmaps and Boxplots:** (Optional) These help visualize pollution patterns and outliers over time.

Integration and Workflow Summary

- **PySpark** handles large-scale data ingestion and transformation efficiently.
- **Pandas and Seaborn** offer high-quality plotting for deeper visual insights.
- The modular approach ensures scalability, as more data or locations can be added seamlessly without reworking the structure.
- Each component—from reading to visualization—plays a role in uncovering patterns, trends, and anomalies in Delhi's air quality data.

Conclusion:

This Air Quality Analysis project successfully demonstrates the use of big data tools in real-world environmental applications. With PySpark, we handled and processed large-scale AQI data efficiently. The analysis revealed crucial pollution patterns in Delhi, supporting the need for timely interventions and public awareness. This project showcases how data science can assist in promoting environmental health and sustainable living.